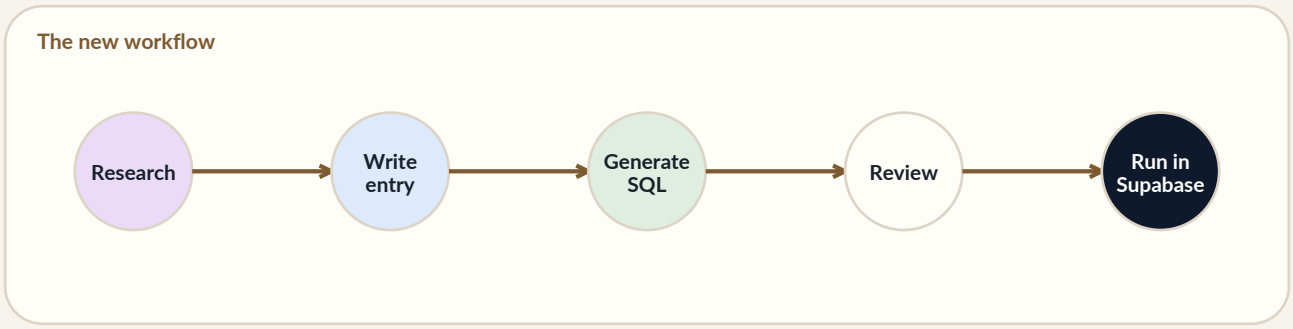
NOMINA

A practical guide to adding names with the Python SQL Builder

Name node | Word node | Concept node | Lineage edge



A reviewable, safe workflow for turning researched name entries into SQL: concepts, word nodes, name nodes, sources, edges, and curated lineages.

Core principle

The Python program does not write directly to Supabase. It writes SQL that you can inspect, edit, and run yourself. Existing nodes are reused by default; existing notes are not overwritten unless you explicitly ask for it.

How this guide is organised

Use this PDF as the working manual for adding names to Nomina using the builder. The guide follows the same visual language as the site: concept nodes, word nodes, name nodes, and highlighted lineages.

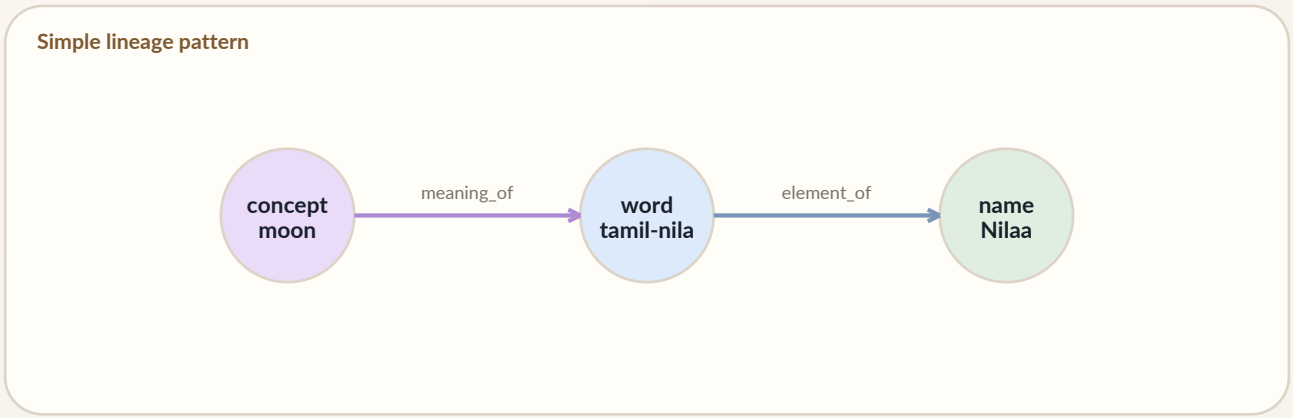
Section	What it covers
1. Mental model	The difference between nodes, detail rows, edges, and lineages.
2. Safe builder behaviour	How the ensure_* functions reuse existing database objects.
3. Simple name	One concept -> one word -> one name.
4. Branching name	Multiple concepts and words feeding one name.
5. Name-to-name lineage	Variants and evolution from one name into another.
6. Sources and languages	How to add general-purpose language rows and source groups.
7. The edit-run-check loop	How to replace the demo section, generate SQL, run it, and verify.
8. Troubleshooting	What to check when something renders oddly or the SQL fails.

Important

The lineage array controls the curated display journey. The edges table controls the actual relationships. Never rely on a grouped lineage to guess pairings. Add explicit edges for every relationship.

1. The Nomina data model

Each visible item in the graph starts as a row in the nodes table. The specialised detail then lives in one of three detail tables: concepts, words, or names.



The most common shape is concept -> word -> name. The actual connection is stored as edges; the curated path is stored in lineages.

Object	Database rows involved	Typical use
Concept node	nodes + concepts	Meaning-level idea such as moon, flower, victory, deity / god, violet / purple.
Word node	nodes + words	Language-specific word or name element such as nilā, dīpa, viola, kallfü.
Name node	nodes + names	Actual display name such as Nilaa, Deepa, Violet, Calfuray.
Edge	edges	Relationship between two nodes, with explanation and certainty.
Lineage	lineages	Curated ordered path for the Show lineage button.

The builder mirrors this model. It has separate functions for concepts, words, names, edges, and lineages so the generated SQL remains easy to inspect.

2. Safe builder behaviour

The builder uses an ensure pattern. This means it looks for an existing row before creating anything new.

```
SELECT id INTO n_example
FROM nodes
WHERE node_type = 'concept'
  AND (
    slug IN ('flower', 'blossom', 'bloom')
    OR lower(canonical_key) IN ('flower', 'blossom', 'bloom')
    OR lower(display_label) IN ('flower', 'blossom', 'bloom')
  )
LIMIT 1;

IF n_example IS NULL THEN
  INSERT INTO nodes (...)
  VALUES (...)
  RETURNING id INTO n_example;
END IF;
```

The corrected builder is deliberately conservative: existing nodes and detail rows are not overwritten by default. That makes it safe to reuse shared nodes like sun, moon, flower, deity / god, Sanskrit dīpa, or Latin viola.

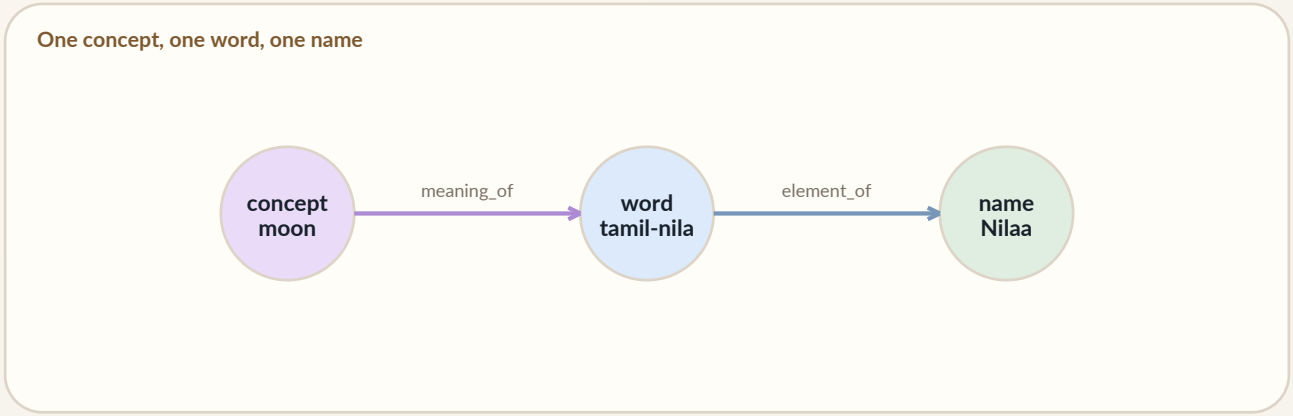
Function	If matching node exists	If matching node does not exist
ensure_concept	Reuse it. Do not rewrite concept details by default.	Create nodes row and concepts row.
ensure_word	Reuse it. Do not rewrite words row by default.	Create nodes row and words row.
ensure_name	Reuse it. Do not rewrite names row by default.	Create nodes row and names row.
ensure_edge	Delete and recreate the specific edge so the current explanation is clean.	Create the edge.
ensure_primary_lineage	Delete and recreate the primary lineage for that name.	Create the primary lineage.

When to update existing content

Set update_existing_node=True and/or update_existing_detail=True only when you intentionally want to revise an existing word/name/concept. For normal name-batch work, leave these as the default False.

3. Adding a simple name

A simple name has one main meaning concept, one source word, and one final name. Nilaa is a good mental model: moon -> Tamil nilā -> Nilaa.



In the Python demo section, this becomes three ensure calls plus two edges and one lineage.

```
b.ensure_concept(ConceptNode(
    slug="moon",
    label="moon",
    canonical_key="moon",
    description_short="The idea of the moon, moonlight, and lunar beauty.",
    description="The idea of the moon, moonlight, and lunar beauty.",
    source_group="nilaa",
))

b.ensure_word(WordNode(
    slug="tamil-nila",
    label="nila / nilaa",
    canonical_key="nila",
    description_short="Tamil word/name element meaning moon or moonlight.",
    language_slug="tamil",
    display_text="nila",
    original_script="nila",
    transliteration="nilā / nila / nilaa",
    word_type="noun / name element",
    literal_meaning="moon, moonlight",
    grammar_notes="Tamil noun used directly as a given name form.",
    additional_notes="Nilaa is a lengthened English spelling preserving the long ā
        vowel.",
    source_group="nilaa",
))

b.ensure_name(NameNode(
    slug="nilaa",
    label="Nilaa",
    canonical_key="nilaa",
    description_short="Tamil feminine given name from nilā, meaning moon or
        moonlight.",
    display_name="Nilaa",
    original_script="nila",
    transliteration="Nilā / Nilaa",
    primary_language_slug="tamil",
    gender_usage="feminine given name",
    short_summary="Nilaa is a Tamil feminine given name meaning moon or
        moonlight.",
    long_summary="Nilaa comes from Tamil nilā, meaning moon or moonlight.",
    literary_notes=None,
    cultural_notes="This is mainly a first/given name rather than a surname.",
    pronunciation_notes="Usually pronounced nee-LAA.",
    certainty_notes="Good certainty for the Tamil nilā meaning.",
    source_group="nilaa",
))
```

Simple name: edges and lineage

After the nodes are ensured, add the relationships explicitly. The edge explanations are valuable because they explain why the relationship exists.

```
b.ensure_edge(Edge(
    from_slug="moon",
    to_slug="tamil-nila",
    edge_type_code="meaning_of",
    certainty=0.96,
    explanation="Tamil nilā means moon or moonlight.",
    source_group="nilaa",
))

b.ensure_edge(Edge(
    from_slug="tamil-nila",
    to_slug="nilaa",
    edge_type_code="element_of",
    certainty=0.94,
    explanation="Tamil nilā forms the name Nilaa.",
    source_group="nilaa",
))

b.ensure_primary_lineage(Lineage(
    name_slug="nilaa",
    title="Etymology of Nilaa",
    summary="Nilaa derives from Tamil nilā, meaning moon or moonlight.",
    path=[
        ["moon"],
        ["tamil-nila"],
        ["nilaa"],
    ],
    certainty=0.94,
    source_group="nilaa",
))
```

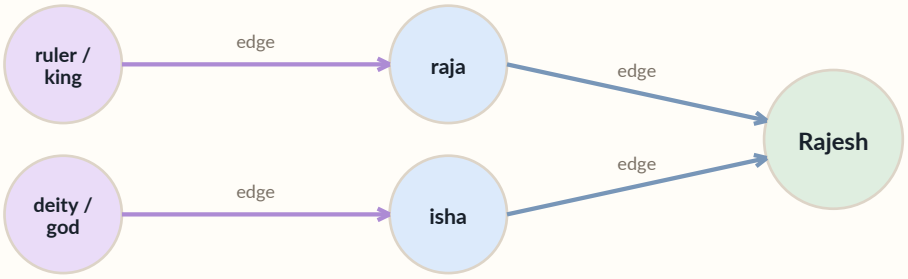
Edge type guide

meaning_of usually connects a concept to a word. element_of connects a word or element to a name. variant_of connects name variants. evolves_to connects a historical form to a later form.

4. Adding a branching name

Some names are compounds or have more than one meaningful route. In these cases, the lineage uses grouped steps, and the edges define the exact pairings.

Parallel lineage groups with explicit edges



The lineage groups show parallel layers. They do not automatically pair ruler / king with raja or deity / god with isha. The explicit edges do that.

```
b.ensure_edge(Edge(
    from_slug="ruler-king",
    to_slug="sanskrit-raja",
    edge_type_code="meaning_of",
    certainty=0.92,
    explanation="Sanskrit raja means king or ruler.",
    source_group="rajesh",
))

b.ensure_edge(Edge(
    from_slug="deity-god",
    to_slug="sanskrit-isha",
    edge_type_code="meaning_of",
    certainty=0.92,
    explanation="Sanskrit isha means lord, ruler, or deity.",
    source_group="rajesh",
))

b.ensure_primary_lineage(Lineage(
    name_slug="rajesh",
    title="Etymology of Rajesh",
    summary="Rajesh combines elements associated with king/ruler and lord/deity.",
    path=[
        ["ruler-king", "deity-god"],
        ["sanskrit-raja", "sanskrit-isha"],
        ["rajesh"],
    ],
    certainty=0.90,
    source_group="rajesh",
))
```

5. Name-to-name lineages

Sometimes the lineage passes from one name to another. This is useful for spelling variants, later language forms, and names that evolved through another name.

Name-to-name route

The Dipika -> Deepika pattern is a good example. Dipika is the closer transliteration; Deepika is a common English spelling that reflects the long ī vowel.

```
b.ensure_edge(Edge(
    from_slug="sanskrit-dipika",
    to_slug="dipika",
    edge_type_code="element_of",
    certainty=0.92,
    explanation="Sanskrit dīpikā forms the name Dipika.",
    source_group="dipika_deepika",
))

b.ensure_edge(Edge(
    from_slug="deepika",
    to_slug="dipika",
    edge_type_code="variant_of",
    certainty=0.90,
    explanation="Deepika is a common English spelling variant of Dipika /
        Dīpikā.",
    source_group="dipika_deepika",
))

b.ensure_primary_lineage(Lineage(
    name_slug="deepika",
    title="Etymology of Deepika",
    summary="Deepika is a spelling variant of Dipika / Dīpikā, associated with
        lamp and light.",
    path=[
        ["lamp-light"],
        ["sanskrit-dipa"],
        ["sanskrit-dipika"],
        ["dipika"],
        ["deepika"],
    ],
    certainty=0.90,
    source_group="dipika_deepika",
))
```

Direction of variant_of

Your current frontend treats variant_of visually as an undirected dashed relationship, but the SQL still stores a from_node_id and to_node_id. Be consistent within a family so the data stays readable.

6. Sources and language nodes

Sources and languages should be added in a general-purpose way. A language node should describe the language, not the one name you are adding today.

Good language note	Avoid
A Romance language descended from Latin, used in Romania, Moldova, and diaspora communities.	Used here for the Viorica violet-name route.
A Japonic language written with kanji, hiragana, and katakana.	Used here for Sumire.
A Semitic language with ancient, liturgical, literary, and modern spoken forms.	Used here for Sigal.

Sources are grouped so multiple entries in one batch can share the same source_ids JSONB variable.

```
b.ensure_sources(
    group_name="violet_family",
    sources=[
        Source(
            title="Behind the Name - Violet",
            author="Behind the Name",
            source_type="name dictionary",
            url="https://www.behindthename.com/name/violet",
            citation_text="Behind the Name, “Violet”.",
            notes="Used for Violet from the English word violet, ultimately
                derived from Latin viola.",
        ),
        Source(
            title="Behind the Name - Ianthe",
            author="Behind the Name",
            source_type="name dictionary",
            url="https://www.behindthename.com/name/ianthe",
            citation_text="Behind the Name, “Ianthe”.",
            notes="Used for Ianthe from Greek ion, violet, and anthos, flower.",
        ),
    ],
)
```

7. Research and writing checklist

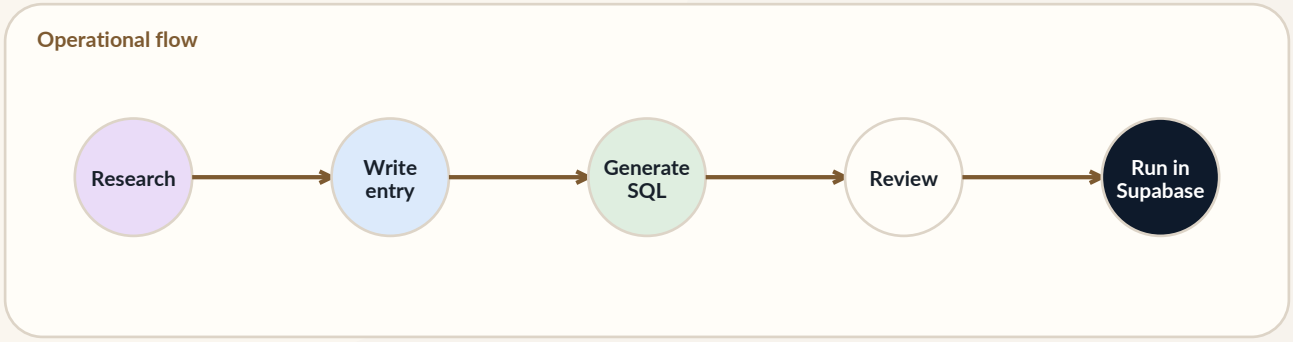
Before writing a new batch, fill this checklist. It keeps the data clean and avoids duplicate nodes.

Question	Why it matters
Is this a first name, surname, both, or uncertain?	Put this in gender_usage and cultural_notes.
What language is the word node in?	The name display language and source word language can differ.
What existing concept nodes can be reused?	Avoid duplicates such as greatness vs. greatness / glory.
Does the name have multiple routes?	Use grouped lineage steps and explicit edges.
Does it pass through another name?	Add a name-to-name variant_of or evolves_to edge.
How certain is each relationship?	Use edge certainty and certainty_notes honestly.
Are sources strong enough?	Prefer reliable name dictionaries, language dictionaries, and etymology references.

Notes style

Write notes about the name itself: meaning, usage, cultural context, spelling variation, and uncertainty. Do not say “the graph uses…” or “this node connects to…” inside name notes. That belongs in edge explanations and lineage summaries.

8. The edit-run-check loop



Only the demo/main section at the bottom needs to change for each batch. The builder classes above it can stay the same.

```
# 1. Replace the demo section at the bottom of nomina_sql_builder.py
# 2. Run the builder
python nomina_sql_builder.py

# 3. Open the generated .sql file and skim:
#    - language rows
#    - sources
#    - node ensure blocks
#    - edges
#    - lineage path_nodes
#    - quick checks

# 4. Run the SQL in Supabase
# 5. Search the name in the Nomina site
# 6. Click Show lineage and verify the visual route
```

The generated SQL ends with quick checks. These are not part of the data write; they simply show what was created or reused.

```
-- Quick check: lineages
SELECT
  n.slug,
  l.title,
  l.summary,
  l.path_nodes,
  l.certainty
FROM lineages l
JOIN nodes n ON n.id = l.node_id
WHERE n.slug IN ('calfuray')
  AND l.lineage_type = 'primary'
ORDER BY n.slug;
```

9. Pattern library

Use these patterns as templates when deciding how to model a name family.

Pattern	Example shape	Use when
Direct word name	moon -> nilā -> Nilaa	The name is directly the word or a simple spelling of it.
Compound name	good + girl -> jó + leány -> jóleán -> Jolánka	The name is built from multiple elements.
Multiple meanings into one word	strength + victory -> vikrama -> Vikram	A word carries several core meanings.
Name variant	Dipika -> Deepika	One name is a spelling/language variant of another.
Historical evolution	Viatrix -> Beatrix -> Beatrice	One form develops into a later form.
Uncertain route	Yolanda -> Jolan and jóleán -> Jolánka -> Jolan	Two plausible explanations should be represented cautiously.

Combined lineages

A primary lineage can include multiple branches when a name has more than one useful explanation. Keep certainty lower and explain the uncertainty in the name notes and edge explanations.

10. Troubleshooting

Problem	Likely cause	Fix
SQL says a variable is unknown	A lineage or edge references a slug that was not ensured earlier.	Add ensure_concept, ensure_word, or ensure_name before the edge/lineage.
Lineage validation fails	Adjacent lineage groups have no explicit edge between them.	Add the missing edge. Do not rely on grouped lineage to imply a relationship.
Existing node was reused unexpectedly	Lookup aliases matched an existing slug, label, or canonical_key.	Check lookup_slugs and lookup_labels. Remove broad aliases if you need a separate node.
Frontend does not show the route	The UI currently loads primary lineage only, or the lineage path omits a node.	Check lineage_type and path_nodes in the quick check.
Nodes appear but edges do not	The frontend did not load all adjacent edges or the edge direction/slugs differ.	Search directly for the name, then Show lineage; inspect edge quick check.
Text looks too graph-focused	Name notes mention implementation details.	Move relationship wording to edge explanations. Keep names notes human and descriptive.

When in doubt, run the generated SQL on a small batch first. One clean name family is easier to debug than twenty names at once.

11. Minimal batch template

This is the shape to copy when starting a new name. Expand it with more concepts, word nodes, names, edges, and lineages as needed.

```
def build_new_name_sql() -> str:
    b = NominaSQLBuilder()
    b.include_edge_types()

    b.ensure_language(Language(
        slug="language-slug",
        name="Language Name",
        alternative_names_json='[]',
        language_family="...",
        period_label="...",
        region_note="...",
        additional_notes="General language overview, reusable for future names.",
    ))

    b.ensure_sources(
        group_name="name_family",
        sources=[
            Source(
                title="Source title",
                author="Source author",
                source_type="name dictionary / dictionary / etymology dictionary",
                url="https://...",
                citation_text="Short citation text.",
                notes="What this source supports.",
            ),
        ],
    )

    b.ensure_concept(ConceptNode(...))
    b.ensure_word(WordNode(...))
    b.ensure_name(NameNode(...))

    b.ensure_edge(Edge(...))
    b.ensure_primary_lineage(Lineage(...))

    return b.render()


if __name__ == "__main__":
    sql = build_new_name_sql()
    output = Path("nomina_batch_new_name.sql")
    output.write_text(sql, encoding="utf-8")
    print(f"Wrote {output}")
```

Best batch size

Three to five names per batch is a good working rhythm. It keeps the SQL reviewable while still making progress quickly. Larger families can be split into base family first, then extension batches.